WBS CODING SCHOOL

WheatIQ
Understanding Life’s Code

Sanaz Arianmanesh

Data Science

April 2026

Why WheatIQ?




- Wheat is a major global food crop
- Diseases reduce yield & quality
- Early detection is challenging
- Farmers lack fast diagnostic tools

How WheatIQ Works

User Interface

Upload or capture a wheat leaf image

Prediction Pipeline

Image processing:

- Wheat detector
- Healthy vs Diseased classifier
- Disease type classifier

Conversational AI

Explains the prediction and provides guidance

Dataset Overview




- ~14,000 wheat leaf images (11 classes)
- Healthy & diseased samples
- ~8,000 non-wheat images
- Train / Validation / Test split (per model)

Challenges & Further Enhancements

- **Similar diseases:** Visually very close

  → hard for CNN to distinguish
- **Image quality:** Blur, lighting

  → affects prediction accuracy
- **GPU limitation**

  → slower training

- Improve classification accuracy

  → larger & balanced dataset
- Smarter chatbot

  → full RAG integration
- Scalable system

  → cloud deployment


Thank you!

Appendix

Models & Chatbot Overview

CNN Pipeline (Wheat Disease Detection)

- Wheat vs Non-Wheat (Binary)
- Healthy vs Diseased (Binary)
- Disease Classification (Multi-Class, 10 classes)

LLM Chatbot (Knowledge-Based)

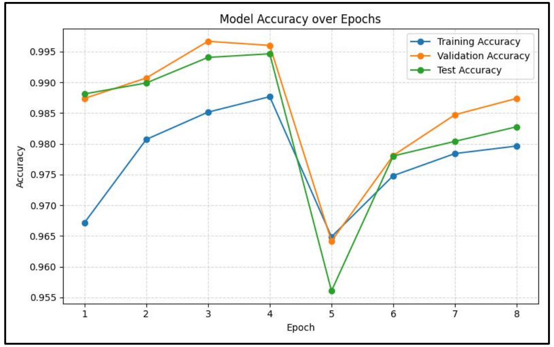
- Groq-based language model
- Uses internal disease knowledge (JSON)
- Provides explanation and treatment advice
- Supports bilingual responses (EN / DE)
- Uses recent chat history for context

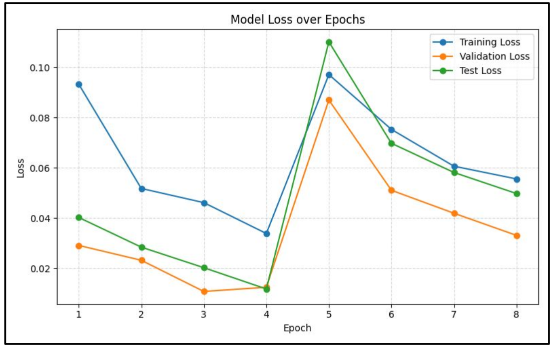
Stage 1: Wheat Detection (Binary Classification)

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
sequential (Sequential)	(None, 224, 224, 3)	0
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
batch_normalization (BatchNormalization)	(None, 1280)	5,120
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 128)	163,968
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 4,218,788 (16.09 MB)
Trainable params: 166,657 (651.00 KB)
Non-trainable params: 4,052,131 (15.46 MB)

Transfer Learning with EfficientNetB0
Optimizer: Adam
Learning rate: 0.001 → 0.00001 during fine-tuning





Model Performance

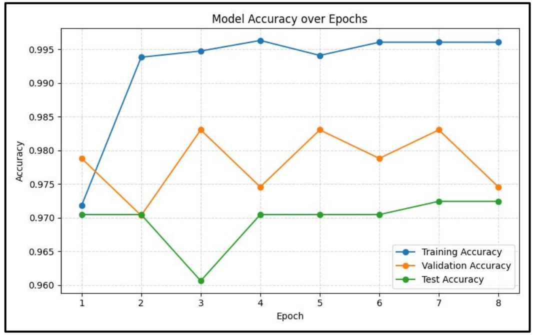
- Test Loss: **0.0202**
- Test Accuracy: **99.41%**

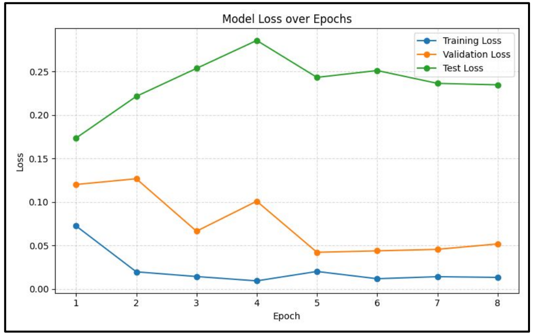
Stage 2: Wheat Health (Binary Classification)

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
sequential (Sequential)	(None, 224, 224, 3)	0
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
batch_normalization (BatchNormalization)	(None, 1280)	5,120
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 128)	163,968
dropout_1 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 4,218,788 (16.09 MB)
Trainable params: 166,657 (651.00 KB)
Non-trainable params: 4,052,131 (15.46 MB)

Transfer Learning with EfficientNetB0
Optimizer: Adam
Learning rate: 0.001 → 0.00001 during fine-tuning





Model Performance

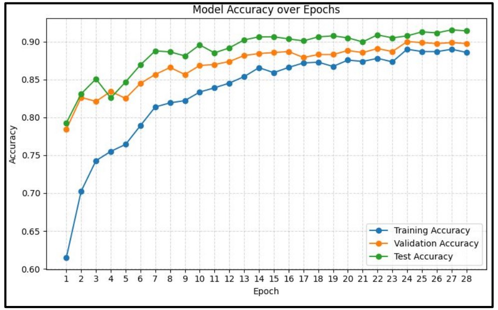
- Test Loss: **0.2536**
- Test Accuracy: **96.06%**

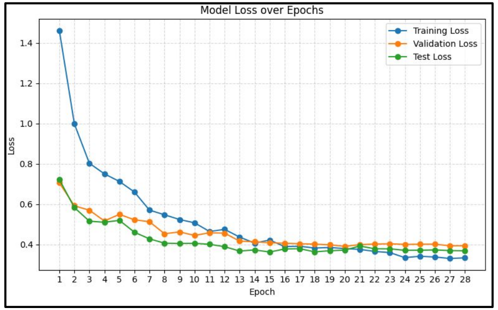
Stage 3: Multi-Class Disease Classification

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 300, 300, 3)	0
sequential (Sequential)	(None, 300, 300, 3)	0
efficientnetb3 (Functional)	(None, 10, 10, 1536)	10,783,535
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1536)	0
batch_normalization (BatchNormalization)	(None, 1536)	6,144
dropout (Dropout)	(None, 1536)	0
dense (Dense)	(None, 256)	393,472
dropout_1 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 11)	2,827

Total params: 11,185,978 (42.67 MB)
Trainable params: 399,371 (1.52 MB)
Non-trainable params: 10,786,607 (41.15 MB)

Transfer Learning with EfficientNetB3
Optimizer: Adam
Learning rate: 0.001 → 0.00001 during fine-tuning

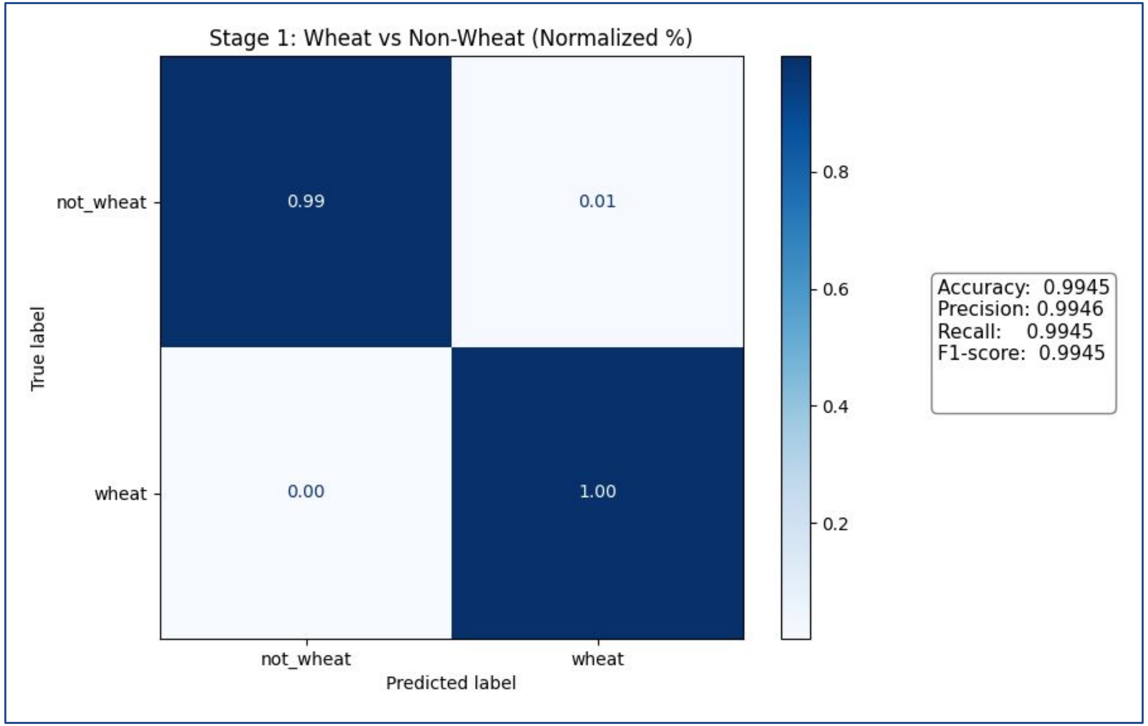




Model Performance

- Test Loss: **0.3718**
- Test Accuracy: **90.5%**

Stage 1: Wheat Detection Model Evaluation

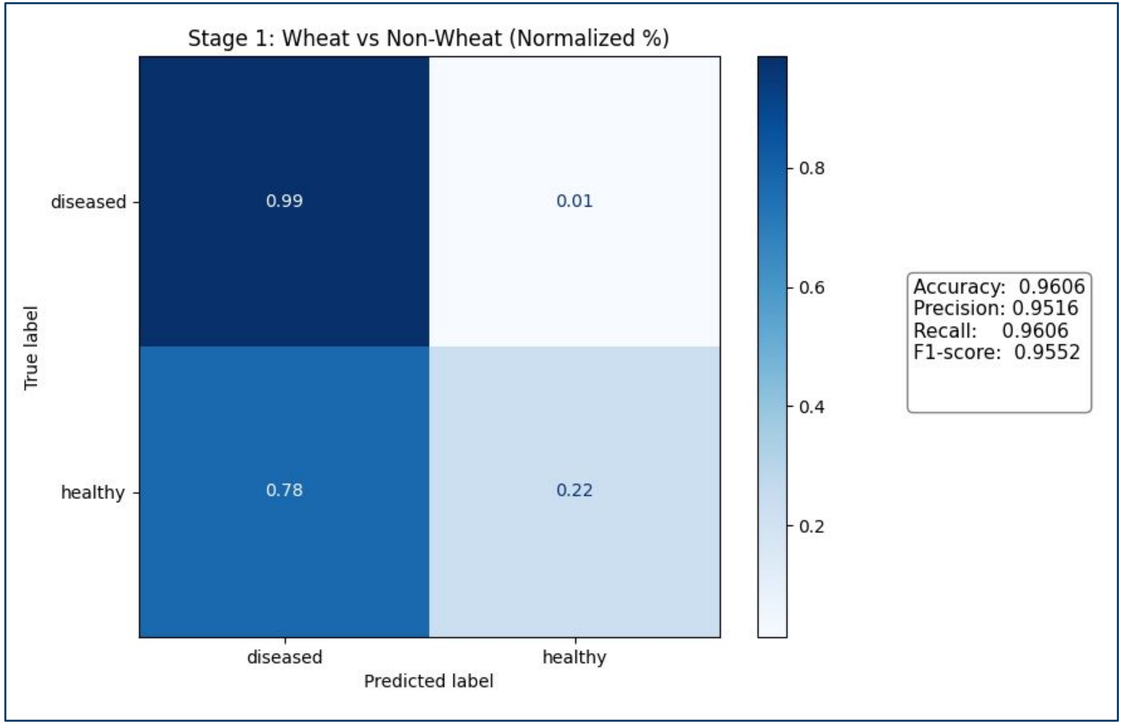


Metrics

- Test Accuracy: **99.45%**
- Precision: **0.9946**
- Recall: **0.9945**
- F1-score: **0.9945**

- **High accuracy** and strong overall performance
- **Near-perfect recall** for wheat (minimal false negatives)
- **High precision** with **very low false positives**
- **Consistent F1-score** with minimal class confusion

Stage 2: Wheat Health Model Evaluation

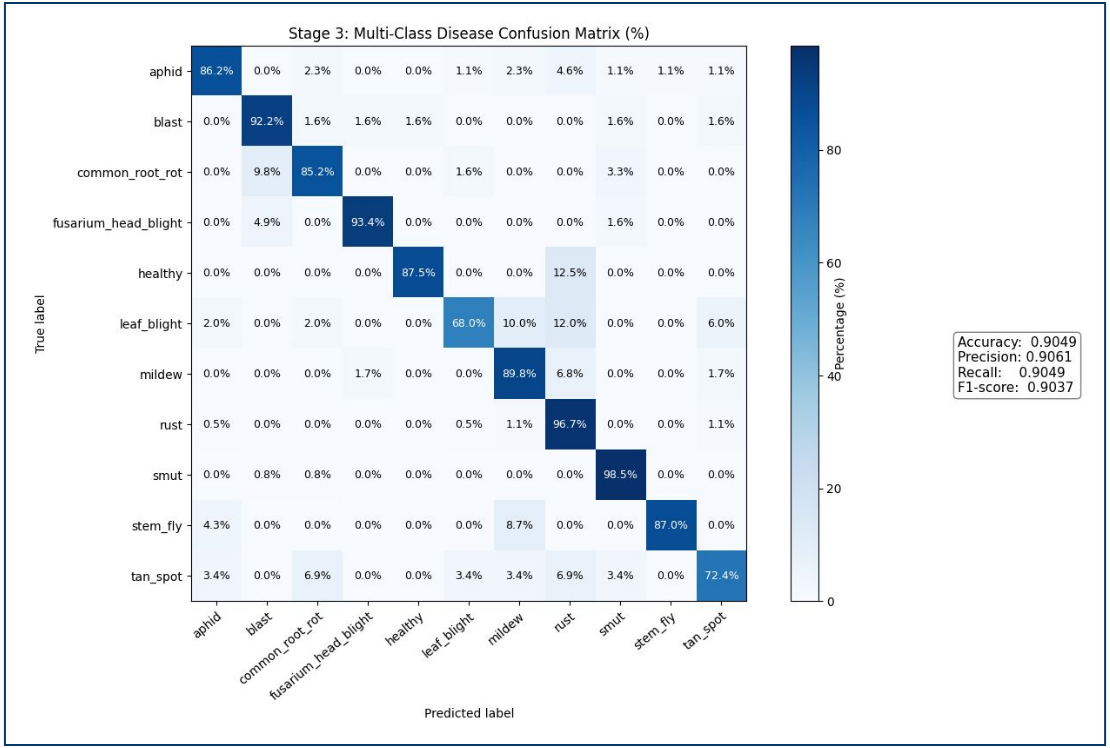


Metrics

- Test Accuracy: **96.06%**
- Precision: **0.9516**
- Recall: **0.9606**
- F1-score: **0.9552**

- **Strong recall** for diseased (critical class)
- Model **prioritizes** detecting disease (low false negatives)
- Trade-off: **lower recall for healthy** (higher false positives)
- **Mild class** imbalance in predictions

Stage 3: Multi-Class Disease Classification Evaluation



Metrics

- Overall Test accuracy: **90.5%**
- Precision: **0.9061**
- Recall: **0.9049**
- F1-score: **0.9037**

- **Strong** overall multi-class performance
- **Balanced precision–recall** across classes
- **Confusion** mainly between **visually similar** diseases
- Performance influenced by **class complexity** and **data size**

Three-stage image inference pipeline

Sequential Stages:

1. Image preprocessing

   (PIL loading, RGB conversion, resizing, NumPy)

2. Wheat detection

   (binary model)

3. Healthy vs Diseased classification

   (binary model)

4. Disease classification

   (multi-class model, top-3)

Additional Logic:

- Threshold-based filtering

  (wheat ≥ 0.80, disease ≥ 0.50)

- JSON label mapping

  (index-to-label from external files)

- Confidence scoring

  (low / moderate / high levels)

- Structured output

  (for UI & chatbot)

Chatbot Architecture

Core Flow:

1. User input

   (question, language, predicted disease)

2. Disease context retrieval

   (load key info from JSON: symptoms, causes, treatment)

3. Prompt construction

   (combine instructions + context + recent messages)

4. LLM response

   (generate answer using Groq API)

Additional Features:

1. Bilingual support

   (English/German output)

2. Chat history context

   (uses previous messages for context)

3. Fallback response

   (works if LLM fails)

4. Controlled output

   (short, practical, no overconfidence)

Disease Classes (Multi-Class Model)

- Aphid
- Blast
- Common Root Rot
- Fusarium Head Blight
- Leaf Blight
- Mildew
- Rust
- Smut
- Stem Fly
- Tan Spot

Libraries & Methods Used

CNN & ML

Libraries

- TensorFlow / Keras
- NumPy
- Matplotlib
- scikit-learn

Methods

- Transfer Learning
- EfficientNetB3/ EfficientNetB0
- Data Augmentation

Backend & UI

Libraries

- Gradio

Methods

- Inference Pipeline
- Image Processing
- User Interaction

LLM Chatbot

Service

- Groq API

Methods

- Prompt Engineering
- Knowledge-based responses
- Chat history
- Bilingual support

Hardware Specs

CPU: Core i5 - 2.40GHz

RAM: 16 GB